SESIÓN N° 3

Tema: Polimorfismo, Clases Abstractas e Interfaces en Java

I. Objetivo de la Guía

Programar la lógica de negocio orientada a objetos creando clases abstractas, aplicando herencia y ejecutando polimorfismo dinámico para gestionar los diversos tipos de recompensas del sistema.

1. Clases Abstractas

Son clases que no se pueden instanciar directamente (no puedes escribir new ItemRecompensa(...)); sirven como plantillas o clases base para otras clases hijas. Pueden contener métodos normales y métodos abstractos (métodos sin cuerpo que obligatoriamente deben ser implementados por las subclases).
Piénsalo como un formulario en blanco: una clase abstracta define QUÉ debe saber hacer cualquier cosa basada en ella (por ejemplo, "todo ítem debe saber aplicarEfecto()"), pero no dice CÓMO — eso lo decide cada subclase. Por eso Java no te deja crear un objeto directo de ella.

2. Polimorfismo

Es la capacidad que tienen los objetos de diferentes clases para responder a un mismo mensaje o método, ejecutando acciones distintas según su propia naturaleza (gracias a la sobrescritura de métodos con @Override). Permite manejar múltiples tipos de objetos bajo una misma referencia común (como un arreglo o lista de la clase padre).
Es como el botón "reproducir" de un control remoto universal: la orden que envías es siempre la misma, pero el televisor, el equipo de sonido o el reproductor de DVD responden a su propia manera al recibirla.

Errores Comunes al Empezar

Error tipico	Como evitarlo
Intentar escribir new ItemRecompensa(...) directamente sobre la clase abstracta.	Las clases abstractas NUNCA se instancian directamente; crea un objeto de una subclase concreta, como new ItemAvatar(...).
Olvidar @Override (o escribir mal el nombre) al sobrescribir aplicarEfecto() en la subclase.	@Override no es obligatorio, pero es tu red de seguridad: si el nombre no coincide EXACTO con el método del padre, el compilador te avisa.
Olvidar el super(...) en el constructor de la subclase cuando la clase padre no tiene un constructor vacío.	Si la clase abstracta tiene un constructor con parámetros, cada subclase debe llamarlo explícitamente con super(...) como primera línea de su propio constructor.

II. Códigos

Parte 1:

Crea una clase abstracta llamada ItemRecompensa.java que servirá como la base para cualquier artículo de la tienda, y al menos dos subclases hijas (ItemAvatar e ItemPaseAcademico). Cada una va en un archivo diferente

- **Codigo 1**

```
// Clase Madre Abstracta (Trabajo 1)
public abstract class ItemRecompensa {
    private String nombre;
    private int costoPuntos;

    public ItemRecompensa(String nombre, int costoPuntos) {
        this.nombre = nombre;
        this.costoPuntos = costoPuntos;
    }
```

```
    // Método abstracto que las subclases deben implementar obligatoriamente
    public abstract void aplicarEfecto();

    // Métodos Getters
    public String getNombre() { return nombre; }
    public int getCostoPuntos() { return costoPuntos; }
}
```

- **Codigo 2:**

```
// Subclase 1: ItemAvatar (Trabajo 1)
public class ItemAvatar extends ItemRecompensa {
    private String tipoSkin;

    public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {
        super(nombre, costoPuntos);
        this.tipoSkin = tipoSkin;
    }

    @Override
    public void aplicarEfecto() {
        System.out.println("Desbloqueando skin de avatar tipo: " + tipoSkin);
    }
}
```

- **Codigo 3**

```
// Subclase 2: ItemPaseAcademico (Trabajo 1)
public class ItemPaseAcademico extends ItemRecompensa {
    private int duracionDias;

    public ItemPaseAcademico(String nombre, int costoPuntos, int duracionDias) {
        super(nombre, costoPuntos);
        this.duracionDias = duracionDias;
    }

    @Override
    public void aplicarEfecto() {
        System.out.println("Otorgando pase académico especial por " + duracionDias + "
días.");
    }
}
```

Explicación Código 1

// Clase Madre Abstracta (Trabajo 1)

Comentario que indica que esta es la clase "madre" (base) abstracta de la que heredarán todos los tipos de recompensa.

public abstract class ItemRecompensa {

Declara la clase ItemRecompensa como abstract: esto impide crear objetos directamente de ella (new ItemRecompensa(...) daría error); solo sirve como plantilla para sus subclases.

private String nombre;
Declara el atributo privado que guardará el nombre del ítem.
private int costoPuntos;
Declara el atributo privado que guardará el costo en puntos del ítem.
public ItemRecompensa(String nombre, int costoPuntos) {
Constructor de la clase abstracta: aunque no se pueda instanciar ItemRecompensa directamente, este constructor SÍ se ejecuta cuando una subclase lo invoca con super(...).
this.nombre = nombre;
Asigna el parámetro nombre al atributo nombre de este objeto.
this.costoPuntos = costoPuntos;
Asigna el parámetro costoPuntos al atributo costoPuntos de este objeto.
}
Cierra el constructor.
// Método abstracto...
Comentario que explica el propósito del método abstracto que sigue.
public abstract void aplicarEfecto();
Declara un método abstracto (sin cuerpo, termina en punto y coma): obliga a que CUALQUIER subclase de ItemRecompensa implemente su propia versión de aplicarEfecto().
// Métodos Getters
Comentario que anuncia los métodos getters.
public String getNombre() { return nombre; }
Método getter que devuelve el nombre del ítem (permite leerlo desde fuera de la clase, ya que el atributo es private).
public int getCostoPuntos() { return costoPuntos; }
Método getter que devuelve el costo en puntos del ítem.
}
Cierra la clase ItemRecompensa.

Explicación Código 2

// Subclase 1: ItemAvatar (Trabajo 1)
Comentario que anuncia el inicio de la primera subclase: ItemAvatar.
public class ItemAvatar extends ItemRecompensa {
Declara la clase ItemAvatar, que hereda (extends) de ItemRecompensa: por eso tiene nombre, costoPuntos y debe implementar aplicarEfecto().
private String tipoSkin;
Declara un atributo propio de ItemAvatar (no existe en la clase base): el tipo de skin.
public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {
Constructor de ItemAvatar, que recibe además el tipo de skin.
super(nombre, costoPuntos);
Llama al constructor de la clase padre (ItemRecompensa) para inicializar nombre y costoPuntos.
this.tipoSkin = tipoSkin;
Asigna el parámetro tipoSkin al atributo propio de esta subclase.
}
Cierra el constructor de ItemAvatar.
@Override
Anotación @Override: le indica a Java (y a quien lea el código) que este método sobrescribe la versión abstracta definida en ItemRecompensa.
public void aplicarEfecto() {
Implementación concreta de aplicarEfecto() específica para ItemAvatar (obligatoria porque la clase base la declaró como abstracta).
System.out.println("Desbloqueando skin...");
Muestra en consola un mensaje propio de este tipo de ítem, usando el atributo tipoSkin.
}
Cierra el método aplicarEfecto() de ItemAvatar.
}
Cierra la clase ItemAvatar.

Explicación Código 3

Subclase 2: ItemPaseAcademico (Trabajo 1)
Comentario que anuncia el inicio de la segunda subclase: ItemPaseAcademico.
public class ItemPaseAcademico extends ItemRecompensa {
Declara la clase ItemPaseAcademico, que también hereda de ItemRecompensa.
private int duracionDias;
Declara un atributo propio de esta subclase: la duración en días del pase.
public ItemPaseAcademico(String nombre, int costoPuntos, int duracionDias) {
Constructor de ItemPaseAcademico, que recibe además la duración en días.
super(nombre, costoPuntos);
Llama al constructor de la clase padre para inicializar nombre y costoPuntos.
this.duracionDias = duracionDias;
Asigna el parámetro duracionDias al atributo propio de esta subclase.
}
Cierra el constructor de ItemPaseAcademico.
@Override
Anotación @Override: indica que este método sobrescribe la versión abstracta de la clase base.
public void aplicarEfecto() {
Implementación concreta de aplicarEfecto() específica para ItemPaseAcademico: distinta a la de ItemAvatar, aunque el método se llama igual.
System.out.println("Otorgando pase académico...");
Muestra en consola un mensaje propio de este tipo de ítem, usando el atributo duracionDias.
}
Cierra el método aplicarEfecto() de ItemPaseAcademico.
}
Cierra la clase ItemPaseAcademico.

Parte 2

Implementación de Polimorfismo

Crea una clase de prueba PrincipalPrueba.java donde se demuestre el uso de un arreglo polimórfico. Guardaremos diferentes tipos de recompensas en una misma lista y llamaremos a sus métodos de forma genérica.

```
public class PrincipalPrueba {
    public static void main(String[] args) {
        // Creación de un arreglo polimórfico de tipo ItemRecompensa (Trabajo 2)
        ItemRecompensa[] inventarioTienda = new ItemRecompensa[2];

        inventarioTienda[0] = new ItemAvatar("Skin Guerrero Cyber", 150, "Legendario");
        inventarioTienda[1] = new ItemPaseAcademico("Pase de Prórroga", 300, 7);

        System.out.println("--- EJECUTANDO EFECTOS CON POLIMORFISMO ---");

        // Recorriendo el arreglo e invocando el método de forma polimórfica (Trabajo 2)
        for (ItemRecompensa item : inventarioTienda) {
            System.out.println("Ítem: " + item.getNombre() + " | Costo: " +
item.getCostoPuntos() + " pts");
            item.aplicarEfecto(); // Comportamiento dinámico según la subclase
            System.out.println("-------------------------------------------------");
        }
    }
}
```

EXPLICACIÓN LÍNEA POR LÍNEA — CÓDIGO 4

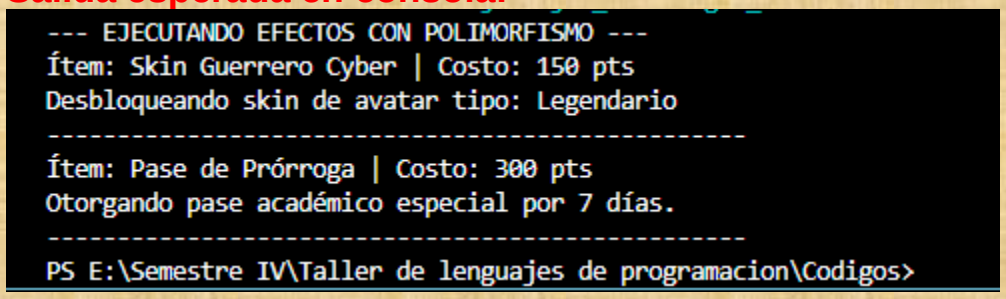
public class PrincipalPrueba {
Declara la clase PrincipalPrueba, donde se probará el polimorfismo (esta clase no hereda de ItemRecompensa, solo la usa).
public static void main(String[] args) {
Punto de entrada del programa: aquí comienza la ejecución cuando se corre esta clase.
// Creación de un arreglo polimórfico...
Comentario que explica qué es un arreglo polimórfico y para qué se usa a continuación.
ItemRecompensa[] inventarioTienda = new ItemRecompensa[2];
Declara un arreglo de tipo ItemRecompensa (la clase abstracta) con espacio para 2 elementos: aunque el tipo declarado es el padre, cada casilla podrá guardar cualquier subclase.
inventarioTienda[0] = new ItemAvatar(...);
Guarda un objeto ItemAvatar en la primera posición del arreglo, aunque el arreglo es de tipo ItemRecompensa: esto es posible porque ItemAvatar ES-UN ItemRecompensa (herencia).
inventarioTienda[1] = new ItemPaseAcademico(...);
Guarda un objeto ItemPaseAcademico en la segunda posición del arreglo: un tipo completamente distinto, pero también válido porque también extiende ItemRecompensa.
System.out.println("--- EJECUTANDO EFECTOS...");
Muestra en consola un encabezado antes de comenzar el recorrido.
// Recorriendo el arreglo...
Comentario que explica que el siguiente bucle es donde ocurre el polimorfismo.
for (ItemRecompensa item : inventarioTienda) {
Recorre el arreglo con un for-each: en cada vuelta, item es tratado como un ItemRecompensa genérico, sin importar si en realidad es un ItemAvatar o un ItemPaseAcademico.
System.out.println("Ítem: " + item.getNombre()...);
Muestra en consola el nombre y el costo del ítem actual, usando los métodos getters heredados de ItemRecompensa.
item.aplicarEfecto();
Llama a aplicarEfecto() sobre item: aquí ocurre el polimorfismo real — Java ejecuta automáticamente la versión de ItemAvatar o de ItemPaseAcademico según el tipo real del objeto, no según el tipo declarado del arreglo.
System.out.println("----------");
Muestra una línea separadora en consola entre un ítem y el siguiente.
}
Cierra el bucle for-each.
}
Cierra el método main.
}
Cierra la clase PrincipalPrueba.

Antes de seguir:

Si guardas un ItemAvatar y un ItemPaseAcademico en el mismo arreglo de tipo ItemRecompensa[] y llamas a aplicarEfecto() en cada uno dentro de un for, ¿el mensaje impreso será igual o distinto para cada uno?

Respuesta: Distinto. Aunque ambos objetos se recorren a través de la misma referencia ItemRecompensa, cada uno ejecuta SU PROPIA versión de aplicarEfecto(). Eso es exactamente el polimorfismo en acción: mismo método, comportamiento distinto según el objeto real.

Salida esperada en consola:



Resumen de Conceptos Clave

- Clase abstracta: Plantilla que no se puede instanciar; obliga a las subclases a implementar métodos abstractos.
- Herencia (extends): La clase hija reutiliza atributos y métodos de la padre y puede agregar los suyos.
- Polimorfismo: Mismo método (aplicarEfecto()) se comporta distinto según el tipo real del objeto.
- @Override: Anotación que confirma al compilador que estás sobrescribiendo un método del padre.
- super(...): Llama al constructor de la clase padre desde el constructor de la hija.